

Search & Indexing Basics

Phase 2 · Intermediate · Estimated time: 1.5 hours

1. Concept From First Principles

Phase 1 covered database indexing briefly — a B-tree index letting you find rows by an exact or range match on a column quickly. But "find all products where the title contains the word 'wireless'" is a fundamentally different problem that a normal B-tree index can't solve efficiently, because the word could be anywhere within the text, in any of millions of rows. This chapter covers the data structure and systems purpose-built for this: full-text search, and the **inverted index** that makes it fast.

A regular database index maps a value to the row(s) containing it — efficient because it's built for exact lookups on a whole field. An **inverted index** flips the relationship for text: instead of mapping documents to their words, it maps individual **terms** (words) to the list of documents that contain them, called a **postings list**. Searching for "wireless" becomes a direct lookup: find the term "wireless" in the index, and you immediately get the list of every document containing it, without scanning a single row of the underlying data. This is precisely why it's called "inverted" — it inverts the natural document-to-words relationship into a words-to-documents relationship.

Backend engineer analogy

Think of the index at the back of a textbook. You don't flip through every page looking for "polymorphism" — you check the index, which lists exactly which pages mention it, and jump straight there. A database's B-tree index on a numeric column works the same way for exact values; an inverted index does the identical job, but for individual words scattered anywhere inside free-form text.

How text becomes an inverted index

Building an inverted index requires a processing pipeline, not just a data structure. **Tokenization** breaks raw text into individual terms (splitting on whitespace and punctuation). **Normalization** lowercases everything and often applies **stemming** (reducing "running," "runs," and "ran" to a common root "run") so that searching for one form matches documents using a different grammatical form. **Stop-word removal** discards extremely common, low-information words ("the," "is," "and") that would otherwise appear in nearly every document and add index size without adding search value. Only after this pipeline does the resulting set of meaningful terms get added to the inverted index, each pointing back to the documents (and often the specific position within each document) where it appears.

Real-world search also needs **ranking**, not just matching — if 50,000 products contain the word "wireless," which ones should appear first? Search engines use scoring algorithms like **TF-IDF** (term frequency – inverse document frequency, favoring documents where a search term appears often, weighted down for terms so common across all documents that they're not very distinguishing) or the more modern **BM25** variant, to rank matching documents by relevance rather than just returning them in an arbitrary order.

In practice, very few teams build an inverted index from scratch — purpose-built search engines like **Elasticsearch** and **OpenSearch** (both built on the Apache Lucene library) provide inverted indexing, tokenization, ranking, and horizontal scaling out of the box, exposed through a simple HTTP/JSON API. Elasticsearch itself scales by sharding its own index across multiple nodes (the same sharding concept from Chapter 6, applied to search indexes rather than relational tables), letting a single logical search index span far more data and query volume than one machine could handle.

A crucial architectural point interviewers probe directly: a search engine like Elasticsearch is almost never your **system of record** — the authoritative, durable copy of your data should live in your primary database (relational or otherwise). Elasticsearch is a derived, read-optimized index built *from* that data, kept in sync via an async pipeline (often exactly the message-queue pattern from Chapter 8: a write to the primary database publishes an event, a consumer updates the search index). This means search results are eventually consistent with the source of truth (Chapter 3), which is almost always an acceptable trade-off for search, and losing the search index entirely is recoverable by rebuilding it from the primary data, which is precisely why you'd never want it to be your only copy.

2. Real-World Example / Case Study

Flipkart's and Noon's product search — typing "wireless bluetooth headphones" and getting relevant, ranked results across a catalog of millions of listings — runs on an Elasticsearch-style inverted index, built from product titles, descriptions, and attributes, kept in sync with the authoritative product-catalog database through an async indexing pipeline rather than being queried directly against the primary database.

Swiggy's and Talabat's restaurant and dish search — searching "biryani near me" and getting relevant restaurants ranked by match quality, distance, and rating — combines full-text search (matching "biryani" against menu items) with geospatial filtering, a common real-world pattern where the inverted index handles the text-matching half of the query while a separate geospatial index or filter handles the location half.

A support or documentation search feature at a company like Razorpay or PhonePe, letting merchants search help articles, is a smaller-scale but equally valid use case, and often a team's first real encounter with search infrastructure — a good illustration that full-text search isn't only for massive consumer product catalogs; it's the right tool any time users need to find free-form text content by approximate or partial word match rather than an exact database field lookup.

3. Diagram

Documents:

Doc 1: "Wireless Bluetooth Headphones"

Doc 2: "Wireless Charging Pad"

Doc 3: "USB Wired Headphones"

Tokenize + normalize + remove stop-words -> build inverted index:

Term	Postings list (which docs contain it)
wireless	-> [Doc 1, Doc 2]
bluetooth	-> [Doc 1]
headphones	-> [Doc 1, Doc 3]
charging	-> [Doc 2]
pad	-> [Doc 2]
usb	-> [Doc 3]
wired	-> [Doc 3]

Search "wireless headphones" -> intersect postings for "wireless" and "headphones" -> [Doc 1, Doc 2] $\cap$ [Doc 1, Doc 3] = [Doc 1] (ranked #1)

4. Trade-offs Table

Approach	Good for	Weak for
Relational DB LIKE '%term%' query	Tiny datasets, occasional simple lookups	Slow at scale; no relevance ranking; full table scans
Inverted index (Elasticsearch/Lucene)	Fast full-text search, relevance ranking, large catalogs	Eventually consistent with source of truth; extra infra to run
Search as system of record	N/A — generally not recommended	Risk of data loss; no ACID guarantees search engines aren't built for
Search as derived, synced index	Best practice: durable primary DB + fast search layer	Requires a sync pipeline (e.g. via message queue)

5. Common Interview Questions

Q1: "Why can't you just use a SQL LIKE '%term%' query for product search at scale?"

Model approach: Explain that LIKE with a leading wildcard can't use a normal B-tree index and forces a full table scan on every query, which doesn't scale past a small dataset, and that it also has no concept of relevance ranking — it's a boolean match, not a scored one, unlike an inverted index built for exactly this purpose.

Q2: "Explain how an inverted index works, from raw text to a fast search query."

Model approach: Walk through tokenization, normalization/stemming, and stop-word removal, then describe how each resulting term maps to a postings list of documents, and how a multi-word query becomes an intersection of postings lists, optionally scored by TF-IDF or BM25 for ranking.

Q3: "Should Elasticsearch be your primary data store for a product catalog?"

Model approach: No — recommend keeping a relational or document database as the system of record, with Elasticsearch as a derived, read-optimized search layer kept in sync asynchronously, explaining that search engines aren't built with the durability/transactional guarantees a primary data store needs.

Q4: "How would you keep a search index in sync with your primary database?"

Model approach: Describe an async pipeline — a write to the primary database publishes a change event (via a message queue, Chapter 8, or a change-data-capture tool), and a consumer updates the search index accordingly, accepting that the search index will be briefly (usually sub-second to a few seconds) eventually consistent with the source of truth.

6. Practice Exercises

- 1 Build a small inverted index by hand for three short product titles of your choosing, then trace through how a two-word search query would use it.
- 2 Explain why stop-word removal (dropping words like 'the' and 'and') makes an inverted index both smaller and more useful for ranking.
- 3 Design the sync pipeline that keeps a restaurant-search Elasticsearch index up to date when a restaurant owner edits their menu in the primary database.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Inverted Indexing Explained Elastic Search & Google Crawlers"	Direct explanation of the core data structure this chapter is built around.
English	"3.4 Elasticsearch Training - Inverted Index Explained"	Hands-on look at how Elasticsearch implements the inverted index internally.
Hindi	"Elasticsearch Tutorial for Beginners (in Hindi)" — playlist	Full Hindi playlist covering Elasticsearch fundamentals step by step.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers search/indexing as part of a full bilingual system design course.

8. Key Takeaways

- An inverted index maps individual words (terms) to the documents that contain them, unlike a normal DB index.
- Building one requires tokenization, normalization/stemming, and stop-word removal before indexing.
- Ranking algorithms like TF-IDF or BM25 order matching results by relevance, not just return arbitrary matches.
- Elasticsearch/OpenSearch (built on Lucene) provide inverted indexing and ranking as a managed service, shardable like Chapter 6.
- A search index should almost always be a derived, synced copy, not your system of record.

- Sync pipelines (often via message queues) keep the search index eventually consistent with the primary database.
- Full-text search is the right tool whenever users need approximate/partial text matching, not just an interview cliché for 'add Elasticsearch'.